

TITOLO

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione



UNIVERSITÀ POLITECNICA DELLE MARCHE

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

RELATORI:

Emanuele Storti

TESINA DI:

Tarek Naja
Sara Vaccaro

A.A 2024/2025

Indice

DESCRIZIONE DEL PROGETTO	1
GLOSSARIO DEI TERMINI	3
GESTIONE DEI REQUISITI	4
Requisiti Funzionali	4
Requisiti Non Funzionali	5
Tabella MoSCoW dei Requisiti	6
Diagrammi dei Casi d'Uso	7
Mockup	9
Verifica e Validazione del Software: Unit Tests	10
Test Manuali	10
Unit Test	10

Descrizione del Progetto

PAWS è un'app che si occupa di facilitare la scoperta e l'eventuale adozione di animali, proponendo un'interfaccia dinamica e interattiva che mostra a schermo un animale alla volta. L'utente, se interessato, può effettuare uno swipe per salvare l'animale in una lista dedicata ai preferiti, altrimenti può semplicemente scorrere per passare al prossimo. Un'ulteriore funzionalità è la "Scheda Dettaglio", una particolare visualizzazione che si attiva con un tap sulla foto: il suo scopo è quello di fornire informazioni approfondite sull'animale, come l'età, la razza, la storia clinica e la posizione geografica. Infine, la sezione "Preferiti" prevede un'area di gestione in cui gli utenti possono consultare in qualsiasi momento gli animali salvati e utilizzare un pulsante dedicato per contattare direttamente il rifugio, l'associazione o l'attuale proprietario.

L'accesso al sistema deve avvenire attraverso delle credenziali che comprendono un indirizzo email e una password di almeno 6 caratteri. Gli animali salvati vengono visualizzati all'interno dei Preferiti in una lista in cui è visibile la foto in miniatura, il nome dell'animale, la razza e la data in cui è stato aggiunto alla lista.

L'app quindi risulta intuitiva e diretta nel proprio utilizzo a seconda delle particolari preferenze e necessità dell'utente; le macro categorie di utilizzo sono:

- Esplorazione dinamica e interattiva tramite meccanica di swipe per scoprire nuovi animali.

- Consultazione informativa attraverso schede dettaglio pensate per conoscere a fondo le caratteristiche e i bisogni di ogni animale.

- Gestione dei Preferiti e contatti, in cui gli utenti possono tenere traccia degli animali di loro interesse e avviare comunicazioni dirette per l'adozione o la richiesta di informazioni.

Sezione

La gestione dei dati rappresenta un elemento centrale del progetto. Il sistema è stato progettato per garantire un'organizzazione chiara e strutturata delle informazioni, supportata da procedure di aggiornamento e validazione che ne assicurano l'affidabilità nel tempo.

Particolare attenzione è rivolta ai principi di integrità e consistenza, così da mantenere i dati coerenti e privi di ridondanze inutili.

In questo modo, la piattaforma garantisce un flusso informativo ordinato, sicuro e facilmente consultabile, a beneficio sia degli operatori interni sia dei clienti.

Architettura del Sistema

L'architettura di GESTRI è concepita secondo un approccio modulare, così da favorire la scalabilità, la manutenzione e l'evoluzione futura della piattaforma. I principali componenti possono essere riassunti come segue:

- **Interfaccia Utente (Front-end):** fornisce un ambiente semplice e intuitivo per l'interazione degli utenti, adattato alle diverse tipologie di attori (Operatori, Staff, Clienti). È progettata per garantire chiarezza, usabilità e accesso rapido alle funzionalità principali.
- **Logica Applicativa (Back-end):** rappresenta il cuore del sistema, gestisce i processi interni e applica le regole di business. Qui risiedono i meccanismi come quelli di registrazione, monitoraggio delle attività, generazione dei documenti e gestione dei permessi degli utenti.
- **Gestione dei Dati (Database):** si occupa della memorizzazione strutturata delle informazioni. La progettazione segue criteri di integrità e consistenza, sfruttando relazioni tra entità per ridurre ridondanze e facilitare le operazioni di ricerca e consultazione.
- **Servizi di Supporto:** includono funzionalità trasversali come autenticazione, controlli di sicurezza e strumenti di reportistica.

Questa architettura a più livelli permette di separare in modo chiaro presentazione, logica e dati, garantendo così robustezza, flessibilità e una più agevole gestione del sistema nel tempo.

Accesso

La gestione delle credenziali assicura una netta distinzione tra i profili e rafforza il livello di sicurezza, prevenendo usi impropri e garantendo che ciascun utente operi esclusivamente nell'ambito delle proprie competenze.

Glossario dei Termini

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Account	Area personale dell'utente.	TECNICO	Profilo
Login	Processo con cui un utente viene identificato e entra nella piattaforma.	TECNICO	Accesso
Sistema	Insieme organizzato di componenti interconnessi che lavorano insieme per svolgere una o più funzioni definite.	TECNICO	Gestionale, Programma, Sito
Utente	Ruolo generico da cui derivano Cliente e Operatore. Ha la capacità di accedere al sistema.	TECNICO	-
Cliente	Soggetto che commissiona il servizio e consulta documenti e stato delle attività.	BUSINESS	-

Gestione dei requisiti

CIAO SARA Nella presente sezione si analizzano in modo approfondito i requisiti del sistema, procedendo alla loro suddivisione secondo le seguenti categorie:

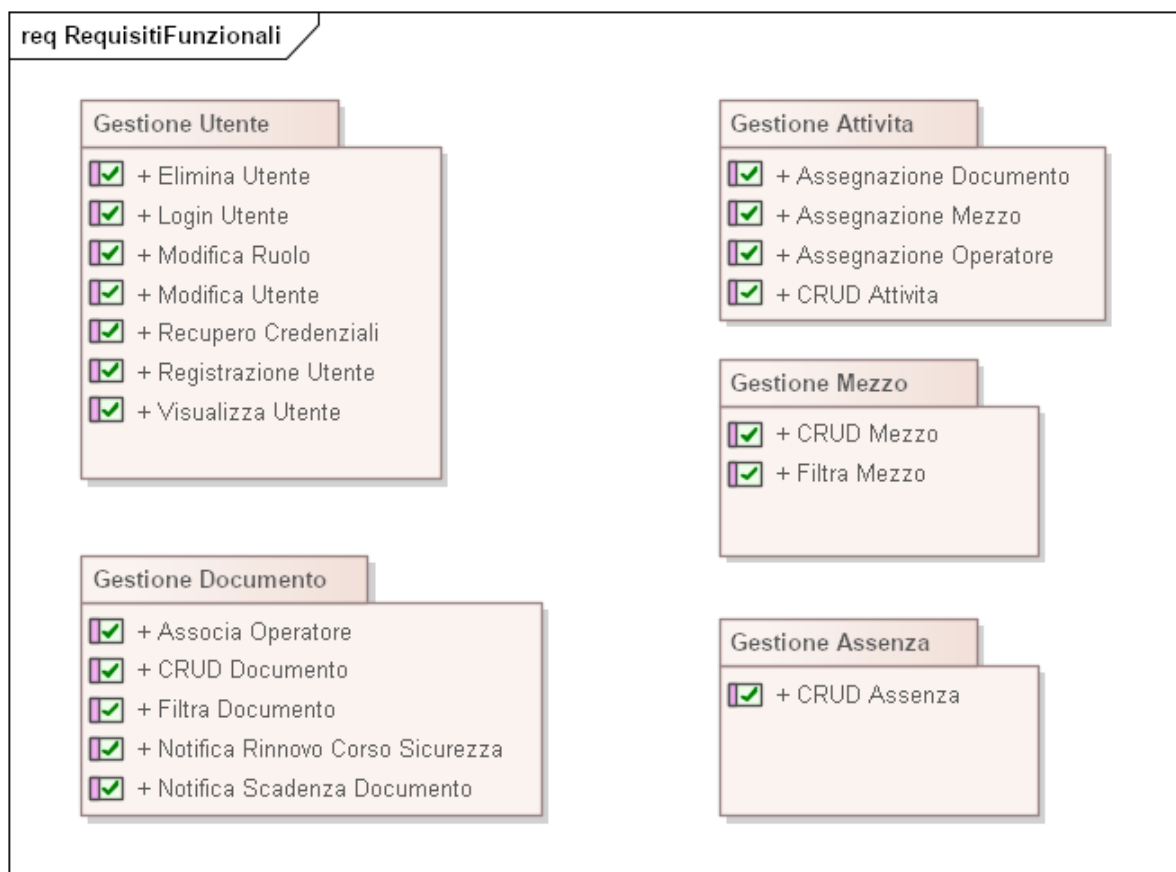
- Requisiti funzionali, che definiscono le specifiche funzionalità che il sistema è tenuto a fornire.
- Requisiti non funzionali, che individuano i vincoli e le qualità che il sistema deve soddisfare, quali ad esempio l'usabilità, la sicurezza e le prestazioni.

Requisiti Funzionali

Area: Gestione Utente

RF1: Registrazione Utente

Il sistema dovrà permettere la registrazione di nuovi utenti (Clienti, Operatori, Staff) inserendo i dati necessari.



Requisiti Non Funzionali

Area: Gestione Tecnologie

RNF1: Implementazione in Python 3

Il sistema dovrà essere implementato utilizzando Python 3.

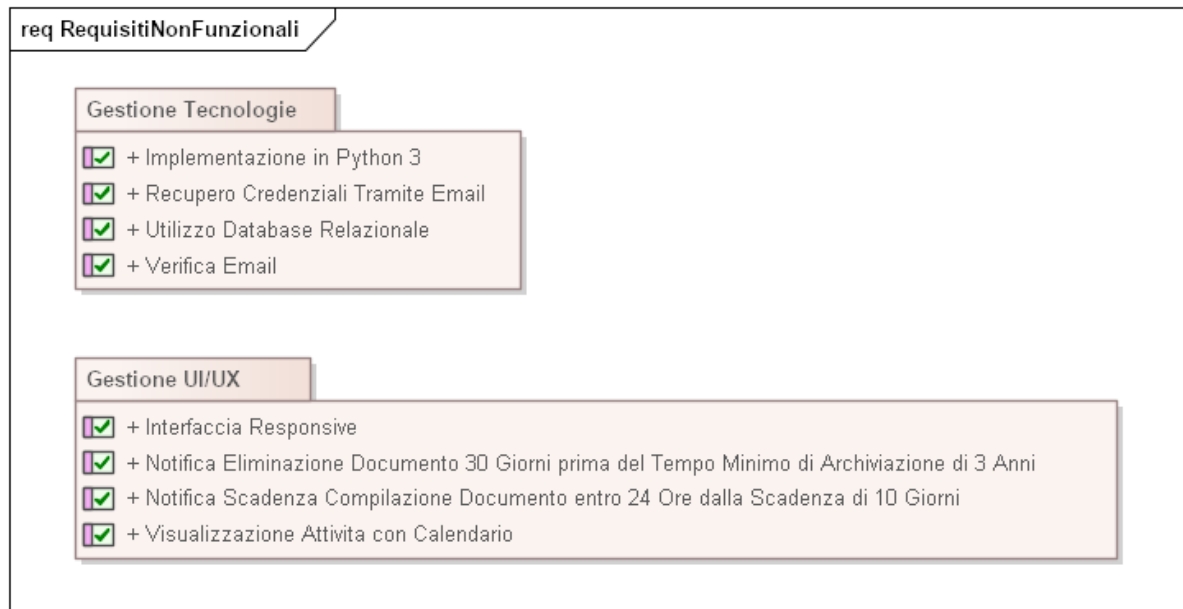


Tabella MoSCoW dei Requisiti

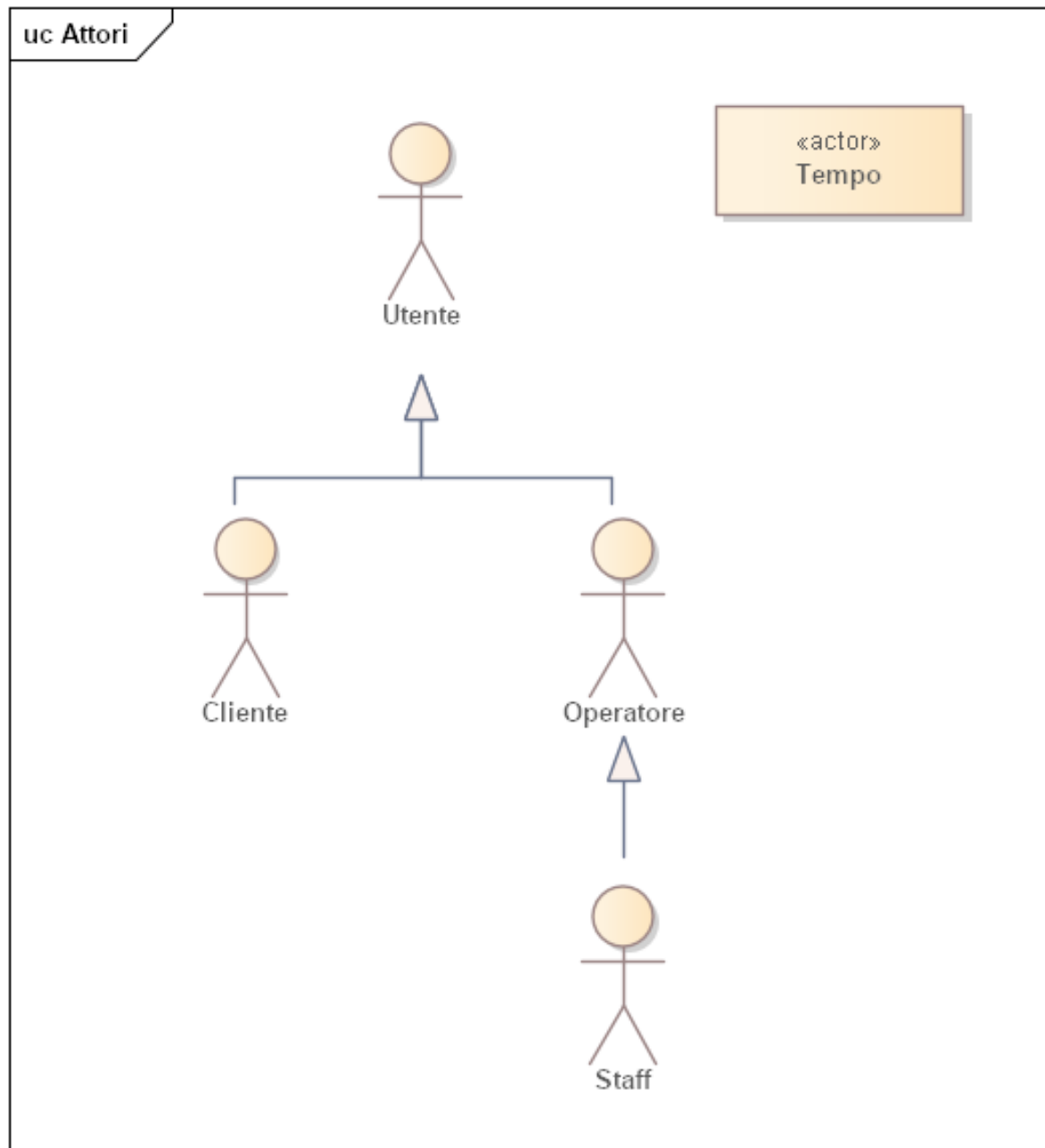
Di seguito la matrice MoSCoW che classifica i requisiti del sistema secondo le priorità: M (Must-have), S (Should-have), C (Could-have), W (Want-have).

Area	Requisito	Priorità MoSCoW
Gestione Utente	RF1: Registrazione Utente	Must
Gestione Utente	RF2: Login Utente	Must

Diagrammi dei Casi d'Uso

Diagramma degli Attori

Il diagramma individua gli attori coinvolti nel sistema e visualizza le connessioni di ereditarietà tra di essi.



Caso d'uso: RegistrazioneUtente	
ID	1
Breve descrizione	Lo Staff o il Cliente si registra nel sistema, con un ruolo che dipende dal punto di accesso
Attori primari	Cliente, Staff
Attori secondari	Nessuno
Precondizioni	Nessuna
Sequenza degli eventi principale	1. If l'Utente è un Cliente che accede alla pagina di registrazione pubblica, allora 1.1. Il sistema mostra il modulo di registrazione 1.2. Il Cliente inserisce le proprie informazioni (nome, cognome, email, password) 1.3. While le credenziali inserite del Cliente non sono valide 1.1. Il sistema richiede al Cliente di inserire le sue informazioni 1.4. Il sistema valida i dati inseriti 1.5. Il sistema crea un nuovo account con il ruolo di Cliente
Postcondizioni	1. È stato creato un nuovo account Utente con ruolo Cliente
Sequenza degli eventi alternativa	1. If l'Utente è un membro dello Staff che accede alla pagina di gestione utenti interna, allora 1.1. Il sistema mostra un modulo di creazione Utente 1.2. Il membro dello Staff inserisce le informazioni del nuovo Utente (nome, cognome, email, password) e seleziona il ruolo desiderato (Operatore o Staff) 1.3. While le credenziali inserite dell'Utente non sono valide 1.1. Il sistema richiede allo Staff di inserire le sue informazioni 1.4. Il sistema valida i dati inseriti 1.5. Il sistema crea un nuovo account con il ruolo specificato (Operatore o Staff)

Mockup

Il mockup dell'applicativo GESTRi è stato realizzato con Figma, uno strumento per la progettazione e la prototipazione di interfacce utente. Ogni mockup rappresenta la versione progettuale dell'interfaccia, definendo struttura, elementi grafici e funzionalità prima della fase di sviluppo.



Figura 1: Mockup login

Verifica e Validazione del Software: Unit Tests

I seguenti test rappresentano l'insieme delle attività di verifica implementate per garantire il corretto funzionamento del progetto. Sono stati adottati due approcci complementari: test automatici lato codice e test manuali delle API.

Test Manuali

Per verificare le API è stata utilizzata l'applicazione Postman, uno strumento che consente di inviare richieste HTTP e analizzare le risposte. Questi test non rientrano nel diagramma degli Unit Test, ma rappresentano un'importante componente di validazione dell'integrazione e del comportamento delle API lato client-server.

Unit Test

La sezione presenta i test unitari eseguiti sul modulo Attività di GESTRi, con l'obiettivo di verificarne la correttezza funzionale e assicurare l'affidabilità del relativo codice.

```
1 class AttivitaIntegrationTests(TestCase):
2     def setUp(self):
3         self.client = Client()
4         # create users
5         self.staff = Utente.objects.create(email='staff@example.com', ruolo=
            Ruolo.STAFF)
6         self.staff.set_password('pass')
7         self.staff.save()
```

Unit Test 1: Attivita